Paul Parisot

Software Engineer - Systems, Simulation & Architecture

Github: paulogarithm — paul.parisot@epitech.eu

About

Systems software engineer specialized in algorithm design, low-level Linux environments, and complex simulation components.
Proven track record in building high-performance architectures (ECS), concurrent sub-systems, and interactive visualization tools.
Analytical problem-solver focused on software craftsmanship, code robustness, and moving progressively into advanced technical architecture.

Skills

Core Programming: C++, C, C# (Unity), Go, Python, Java, x86/ARM Assembly stubs
Systems & Modeling: Data-Oriented Design (ECS), Concurrent Modeling, Multi-threading, IPC (`shm`)
Tooling & Infrastructure: Linux, CMake, GCC, GDB, Valgrind, Docker Engine, Git, Cross-Compilation
Data & Automation: Python Data Automation, Relational Schemas, Code Auditing, Lifecycle Documentation

Professional Experience

Faurecia / Forvia - Software Engineer (Systems & Visualization)

- Developed and optimized native C++ software modules dedicated to hardware driver communication interfaces.
- Engineered interactive C# applications using the Unity engine for real-time subsystem visualization and tracking tools.
- Managed strict resource constraints and memory-safety parameters on target environments.
- Collaborated within an industrial Agile/Scrum environment, interfacing with technical requirements via **Jira**.

Gno.land - Core Software Engineer (Distributed Runtime Engines)

- Refactored core runtime sub-systems, focusing on multi-threaded thread safety and resource-efficient concurrency.
- Isolated and resolved **6 critical execution bugs** (race conditions, memory leaks), committing over 10,000 lines of production code.
- Designed full integration test suites to validate complex system-level protocols and distributed communications.

Epitech / IONIS - Systems Programming Instructor & Code Auditor

- Instructed **400+ students** in advanced C/C++ memory management, low-level pointer mechanics, and Unix system calls.
- Conducted exhaustive code audits to eliminate undefined behaviors, memory corruption, and unhandled system signals.

Architecture & Algorithmic Technical Projects

rodox – Low-Latency Linux Systems Engine

Engineered a modular runtime infrastructure under Linux utilizing a data-oriented layout (ECS) to eliminate CPU cache misses.
Developed custom **static pool allocators** to entirely bypass OS-level heap allocation latencies (`malloc`/`new`).
Implemented high-performance asynchronous networking and communication layers using `Boost.Asio` over TCP. Profiled and hardened code integrity using **GDB** and **Valgrind** to guarantee absolute deterministic execution.

Skills: C++, Boost.Asio, Memory Management, Linux System APIs, Profiling (Valgrind), CMake, Concurrency

Sledge – Containerized Orchestration & Data Core

Architected and developed a complete backend platform in Go optimized for high-volume data stream operations.
Utilized the Docker API at runtime to dynamically spin up, isolate, and orchestrate concurrent environment containers.
Authored automated processing and validation scripts in **Python** to handle schema validation and track pipeline stability.

Skills: Go, Docker API, Python Scripting, Data Integrity, Linux Environments, System Automation

cclass – Runtime Memory Mutation Engine

Designed a pure C framework that dynamically modifies compiled assembly instructions inside raw memory pages.
Utilized `mmap` system calls with precise protection flags to bypass traditional virtual table dispatch overheads.
Enforced strict memory alignment rules to maintain absolute stability across underlying hardware execution targets.

Skills: C, Linux Systems, x86 Assembly, mmap, Memory Safety, Low-level Debugging

Education

Epitech, Paris 2022 – 2027
Master's Degree in Information Technology (Master's level diploma)
Level-7 Certification - Expert in Software Engineering
Specialization: Systems Architecture

McGill University, Montréal 2025 – 2026
Certificate in Management
Technical Leadership & Strategy